

MLOps basics

This is taken from <https://ml-ops.org/content/mlops-principles>

1. Designing the ML-powered Application

- Focus: Business & data understanding, ML solution design.
- Identify users + define ML use-cases (focus on one at a time).
- Goal: Boost user productivity or app interactivity.
- Activities:
 - Assess available data.
 - Define functional/non-functional ML requirements.
 - Design system architecture.
 - Plan model serving strategy.
 - Create ML test suite.

2. ML Experimentation and Development

- Focus: Validate ML feasibility via a Proof-of-Concept.
- Iterative steps:
 - Choose & tune ML algorithms.
 - Data engineering + model engineering.
- Goal: Build a high-quality, production-ready model.

3. ML Operations

- Focus: Deploy and maintain the model in production.
- Use DevOps practices:
 - Testing, versioning, CI/CD, monitoring.

Interconnectivity

- Decisions in early phases (e.g., design) affect experimentation and ops stages.
 - All phases influence each other continuously.
-

After analyzing the MLOps Stages, we might notice that the MLOps setup requires several components to be installed or prepared. The following table lists those components:

MLOps Setup Components	Description
Source Control	Versioning the Code, Data, and ML Model artifacts.
Test & Build Services	Using CI tools for (1) Quality assurance for all ML artifacts, and (2) Building packages and executables for pipelines.
Deployment Services	Using CD tools for deploying pipelines to the target environment.
Model Registry	A registry for storing already trained ML models.
Feature Store	Preprocessing input data as features to be consumed in the model training pipeline and during the model serving.
ML Metadata Store	Tracking metadata of model training, for example model name, parameters, training data, test data, and metric results.
ML Pipeline Orchestrator	Automating the steps of the ML experiments.

All suitable cases where model, data or process needs to be changed (according to [SIG MLOps](#)):

- ML models can be retrained based upon new training data.
- Models may be retrained based upon new training approaches.
- Models may be self-learning.
- Models may degrade over time.
- Models may be deployed in new applications.
- Models may be subject to attack and require revision.
- Models can be quickly rolled back to a previous serving version.
- Corporate or government compliance may require audit or investigation on both ML model or data, hence we need access to all versions of the productionized ML model.
- Data may reside across multiple systems.
- Data may only be able to reside in restricted jurisdictions.
- Data storage may not be immutable.
- Data ownership may be a factor.

Analogously to the best practices for developing reliable software systems, every ML model specification (ML training code that creates an ML model) should go through a code review

phase. Furthermore, every ML model specification should be versioned in a VCS to make the training of ML models auditable and reproducible.

1. Features & Data Tests

- **Data validation:** Auto-check schema/domain via training set stats.
 - **Feature importance:**
 - Compute correlation.
 - Train with individual features.
 - “Leave-one-out” feature tests.
 - Measure inference latency + RAM vs. predictive power.
 - **Remove unused features**, document cleanup.
 - **Policy compliance:** Ensure GDPR, etc. are checked programmatically.
 - **Unit test feature creation code.**
-

2. Model Development Tests

- **Align training with business goals:** Loss metrics (MSE, log-loss) ↔ business metrics like user retention and user happiness.
 - Action: A/B test with degraded models to test alignment.
 - **Model staleness test:**
 - A/B test older models → build Age vs. Quality curve.
 - **Compare model complexity:** Baseline (e.g., linear) vs. advanced models.
 - **Final validation set:** Fully disjoint from training/validation.
 - **Bias/fairness testing:**
 - Collect inclusive data.
 - Inspect correlation with protected attributes.
 - **Conventional unit tests:** For training/specification code.
-

3. ML Infrastructure Tests

- **Reproducibility:**
 - Identify & minimize randomness.

- **Stress/crash tests:**
 - Random input + 1-step train (unit test).
 - Crash → restore from checkpoint.
 - **Training correctness:**
 - Run a few steps → check that loss decreases.
 - **Avoid** diff-testing with past models (hard to maintain).
 - **Integration tests:** Full ML pipeline runs end-to-end.
 - Should pass before production.
 - **Pre-serving validation:**
 - Set thresholds for drift & sudden drops in quality.
 - Canary test: Load model → run predictions on live-like data.
 - Ensure **training vs serving** consistency (same input → same output).
 - Watch for performance drop from holdout → “next-day” data.
-

There is also ML Test Scores -

<https://static.googleusercontent.com/media/research.google.com/en//pubs/archive/aad9f93b86b7addfea4c419b9100c6cdd26cacea.pdf>

Tips & Tricks

Phase	Challenges	How to Ensure Reproducibility
Collecting Data	Generation of the training data can't be reproduced (e.g due to constant database changes or data loading is random)	1) Always backup your data. 2) Saving a snapshot of the data set (e.g. on the cloud storage). 3) Data sources should be designed with time-stamps so that a view of the data at any point can be retrieved. 4) Data versioning.
Feature Engineering	Scenarios: 1) Missing values are imputed with random or mean values. 2) Removing labels based on the percentage of observation. 3) Non-deterministic feature extraction methods.	1) Feature generation code should be taken under version control. 2) Require reproducibility of the previous step "Collecting Data"
Model Training / Model Build	Non-determinism	1) Ensure the order of features is always the same. 2) Document and automate feature transformation, such as normalization. 3) Document and automate hyperparameter selection. 4) For ensemble learning: document and automate the combination of ML models.
Model Deployment	1) Training the ML model has been performed with a software version that is different to the production environment. 2) The input data, which is required by the ML model is missing in the production environment.	1) Software versions and dependencies should match the production environment. 2) Use a container (Docker) and document its specification, such as image version. 3) Ideally, the same programming language is used for training and deployment.

Loosely coupled architecture takeaway:

- Try to **modularize ML projects** (e.g., clean project structure, clear input/output contracts, reusable components).
- Use **templates or standard structures** to help organize your ML code into loosely coupled parts.

The paper specified that teams where the components are less interdependent (or made so that the dependency doesn't cause much hassle, by being very specific with input and output declarations) are more efficient.

ML-based Software Delivery Metrics

Delivery of the model can be sort of scored as well:

Metric	DevOps	MLOps
Deployment Frequency	How often does your organization deploy code to production or release it to end-users?	ML Model Deployment Frequency depends on 1) Model retraining requirements (ranging from less frequent to on-line training). Two aspects are crucial for model retraining 1.1) Model decay metric. 1.2) New data availability. 2) The level of automation of the deployment process, which might range between *manual deployment* and *fully automated CI/CD pipeline*.
Lead Time for Changes	How long does it take to go from code committed to code successfully running in production?	ML Model Lead Time for Changes depends on 1) Duration of the explorative phase in Data Science in order to finalize the ML model for deployment/serving. 2) Duration of the ML model training. 3) The number and duration of manual steps during the deployment process.
Mean Time To Restore (MTTR)	How long does it generally take to restore service when a service incident or a defect that impacts users occurs (e.g., unplanned outage or service impairment)?	ML Model MTTR depends on the number and duration of manually performed model debugging, and model deployment steps. In case, when the ML model should be retrained, then MTTR also depends on the duration of the ML model training. Alternatively, MTTR refers to the duration of the rollback of the ML model to the previous version.
Change Failure Rate	What percentage of changes to production or released to users result in degraded service (e.g., lead to service impairment or service outage) and subsequently require remediation (e.g., require a hotfix, rollback, fix forward, patch)?	ML Model Change Failure Rate can be expressed in the difference of the currently deployed ML model performance metrics to the previous model's metrics, such as Precision, Recall, F-1, accuracy, AUC, ROC, false positives, etc. ML Model Change Failure Rate is also related to A/B testing.

To improve the effectiveness of the ML development and delivery process one should measure the above four key metrics. A practical way to achieve such effectiveness is to implement the CI/CD pipeline first and adopt test-driven development for Data, ML Model, and Software Code pipelines.

Final cheat sheets:

MLOps Principles	Data	ML Model	Code
Versioning	1) Data preparation pipelines 2) Features store 3) Datasets 4) Metadata	1) ML model training pipeline 2) ML model (object) 3) Hyperparameters 4) Experiment tracking	1) Application code 2) Configurations
Testing	1) Data Validation (error detection) 2) Feature creation unit testing	1) Model specification is unit tested 2) ML model training pipeline is integration tested 3) ML model is validated before being operationalized 4) ML model staleness test (in production) 5) Testing ML model relevance and correctness 6) Testing non-functional requirements (security, fairness, interpretability)	1) Unit testing 2) Integration testing for the end-to-end pipeline
Automation	1) Data transformation 2) Feature creation and manipulation	1) Data engineering pipeline 2) ML model training pipeline 3) Hyperparameter/Parameter selection	1) ML model deployment with CI/CD 2) Application build
Reproducibility	1) Backup data 2) Data versioning 3) Extract metadata 4) Versioning of feature engineering	1) Hyperparameter tuning is identical between dev and prod 2) The order of features is the same 3) Ensemble learning: the combination of ML models is same 4) The model pseudo-code is documented	1) Versions of all dependencies in dev and prod are identical 2) Same technical stack for dev and production environments 3) Reproducing results by providing container images or virtual machines
Deployment	1) Feature store is used in dev and prod environments	1) Containerization of the ML stack 2) REST API 3) On-premise, cloud, or edge	1) On-premise, cloud, or edge
Monitoring	1) Data distribution changes (training vs. serving data) 2) Training vs serving features	1) ML model decay 2) Numerical stability 3) Computational performance of the ML model	1) Predictive quality of the application on serving data

Along with the MLOps principles, following the set of best practices should help reducing the "technical debt" of the ML project:

MLOps Best Practices	Data	ML Model	Code
Documentation	1) Data sources 2) Decisions, how/where to get data 3) Labelling methods	1) Model selection criteria 2) Design of experiments 3) Model pseudo-code	1) Deployment process 2) How to run locally
Project Structure	1) Data folder for raw and processed data 2) A folder for data engineering pipeline 3) Test folder for data engineering methods	1) A folder that contains the trained model 2) A folder for notebooks 3) A folder for feature engineering 4) A folder for ML model engineering	1) A folder for bash/shell scripts 2) A folder for tests 3) A folder for deployment files (e.g Docker files)

Recommended Team Composition

Role	Responsibilities	% in Team
Data scientist	Modeling, testing, optimization	20
Data engineer	Data wrangling, feature engineering, ETL, storage	30
Software engineer	Developing services, APIs, user interfaces, integrations	20
ML engineer	Model productization, scaling, packaging, integrations	20
Operations	Infrastructure, deployment, operations	10

Team Building

- Understand the budgetary constraints for the project
- If small team size, look for engineers with multiple skills
- Focus initially on data engineering and data science;
add others as models deliver reasonable results
- Include engineers with domain expertise
- One or two engineers/leads should focus on E2E integration
- Cross-train and document

Implementing Data Governance

- Identify a training data governance team
 - Owners
 - Custodians
- Create policies, processes, and procedures
 - Reuse production data policies as applicable

Question 3 of 3

What are DVC and Neptune examples of?



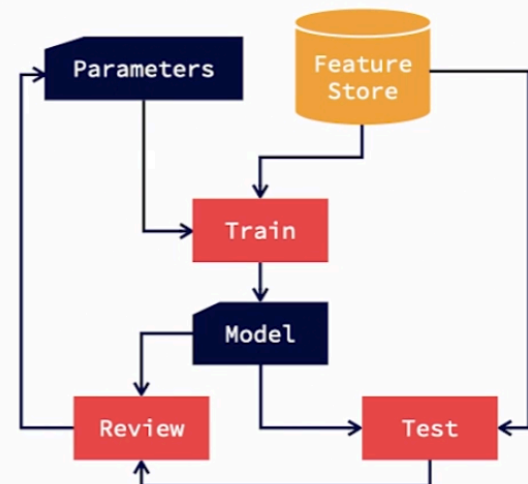
data versioning tools

Correct

Another example of a data versioning tool is lakeFS.

Training Pipeline Functions

- Get training dataset from feature store
- Get parameters for training
- Build and validate model
- Get test data from feature store
- Test the model and review results
- Update parameters and repeat



AutoML Benefits and Shortcomings

- Benefits

- Efficiency
- Speed
- Better results (maybe)
- Accessibility
- Models by customer

- Shortcomings

- New use cases may create errors
- Bias may go untracked
- Processing capacity
- Explainable AI made worse



Up next
Model versioning
1m 22s



Question 1 of 4

Chen and his team are considering which tools to use for their new foray into MLOps. Which popular tool should they consider for experiment management?

☐ Databricks

☒ Kubeflow
Correct

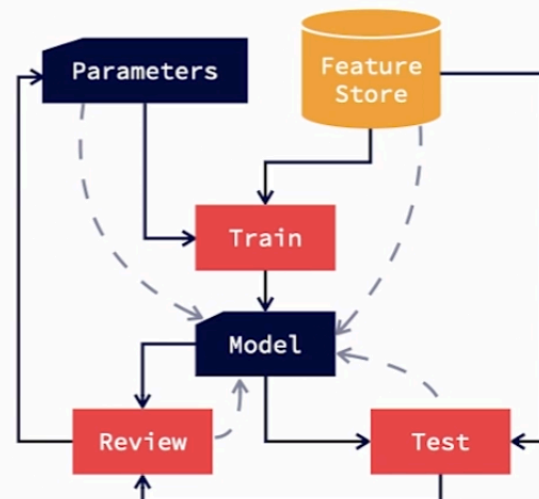
Other popular experiment management tools are MLflow and Weights & Biases.

☐ Jupyter Notebooks
Incorrect

Jupyter Notebooks is better suited for MLOps development.

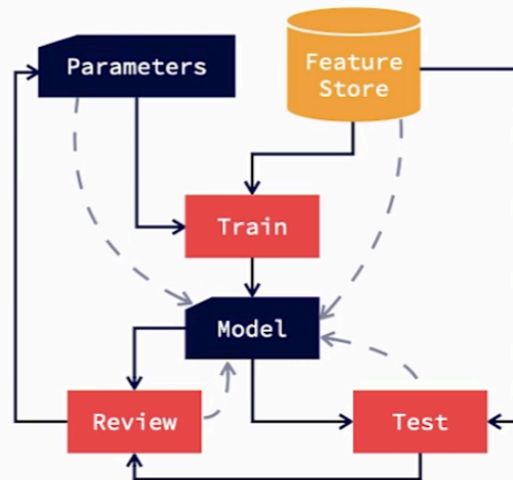
Model Registry Contents

- Serialized model
- Model ID, version, description
- Hyperparameters
- Model status
- Model history

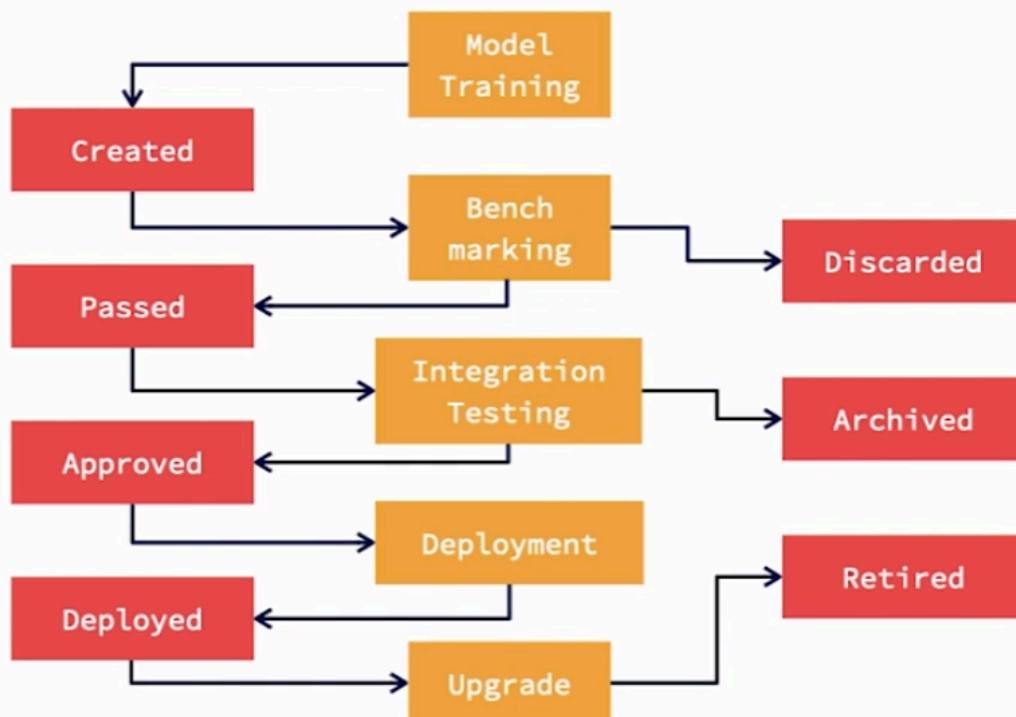


Model Registry Contents

- Link to:
 - Input data versions
 - Code/notebook versions
 - Performance results



Sample Model (Version) Life Cycle



Model Promotion Guidelines

- Create life cycle and promotion criteria during requirements stage
 - Performance and operational metrics
 - Integration and testing – pass criteria
- Automate promotions in the pipeline
- Add human oversight as needed
- Capture decision history in model registry

Converting Notebook to Software

- Preprocessing of input data
- Model file integration—from model repo
- Prediction
- Post-processing—convert raw predictions to business outcomes
- Interfaces for input/output
- Build, test, and deploy infrastructure